FABRIC ATLAS

# Fabric Atlas turns Fabric metadata into a shared working catalog.

## Development, architecture and implementation choices

React + Rayfin Functions · Fabric SQL · minimal Python compatibility

**Connect → Discover → Collect → Validate → Publish**

13 slides · October 2026 · Fabric Atlas 2.1.2

MICROSOFT FABRIC GOVERNANCE

# Fabric Atlas

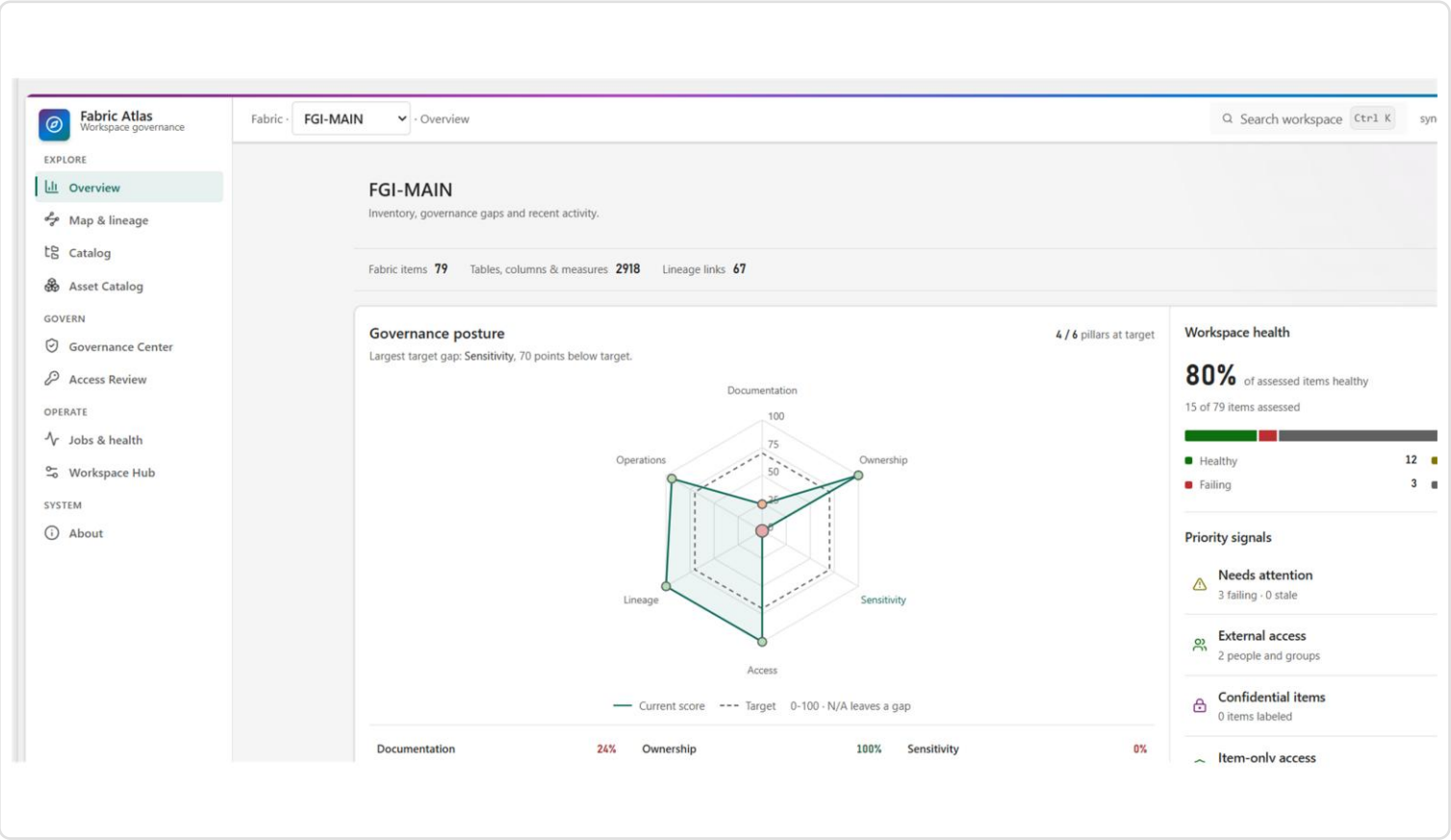
## Workspace intelligence for Microsoft Fabric.

Catalog · lineage · governance · access · operations

OPEN SOURCE MIT LICENSED FABRIC NATIVE

PRODUCT

# One validated snapshot powers every selected workspace view.

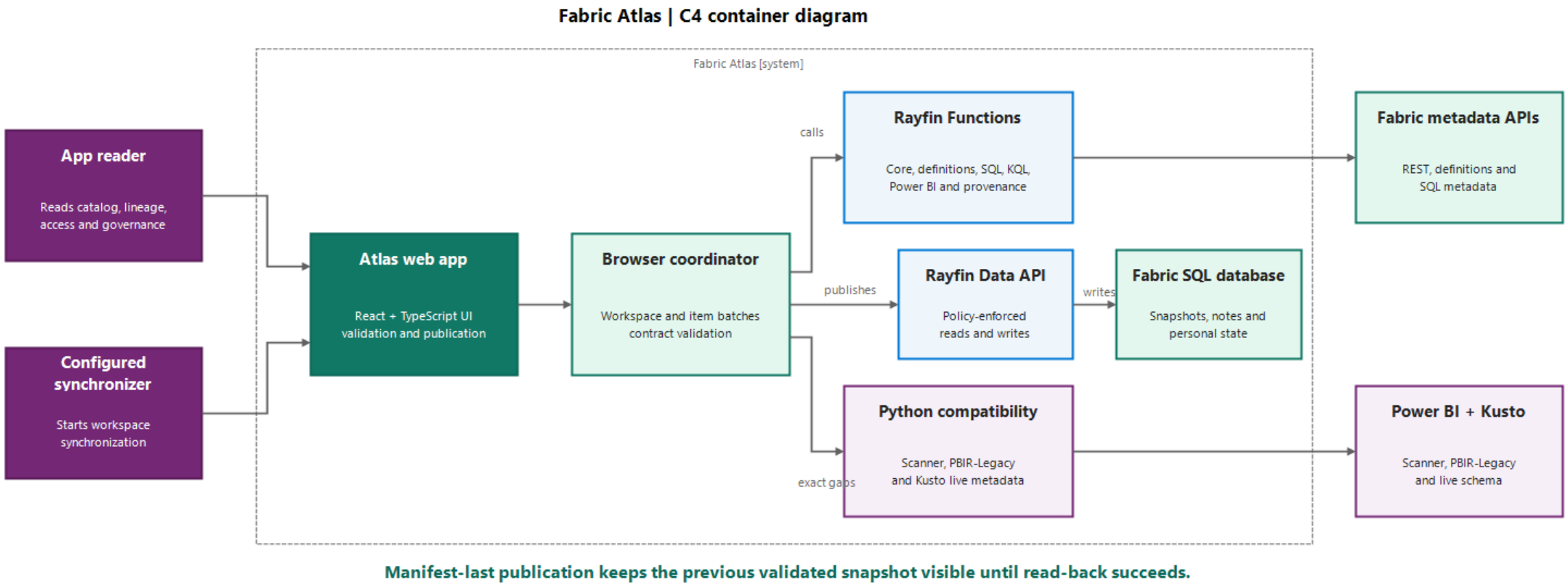



- Multi-workspace catalog, persistent sync status and switching.
- Catalog, object lineage, governance and recent jobs.
- Semantic X-Ray, Item Relations Preview and evidence-based Departure Packs.
- Pages read the validated snapshot; they do not rescan Fabric.

Fabric Atlas 2.1.2 Overview; workspace figures are live evidence, not defaults.

Source: docs/screenshots/workspace-overview-v2.png

# Rayfin Functions collect. The browser validates and publishes.




**Python remains only for the Power BI scanner, PBIR-Legacy and Kusto gaps.**

Source: docs/architecture.md

SYNCHRONIZATION PROCESS

# Synchronization follows five stages and publishes the manifest last.

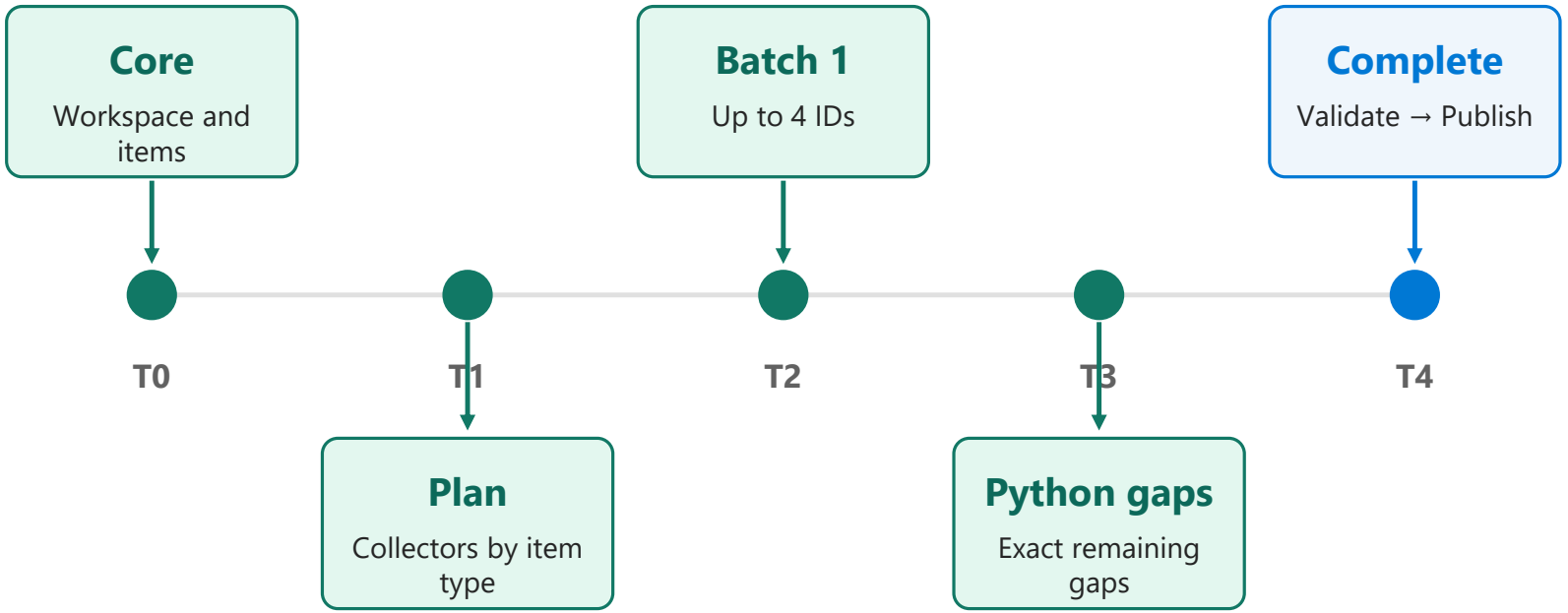



**The previous snapshot stays visible until the Workspace manifest is created.**

Source: src/atlas/browser-collector-sync.ts · src/atlas/backend.ts

BOUNDED COLLECTION

# Rayfin Functions process large workspaces without restarting the run.

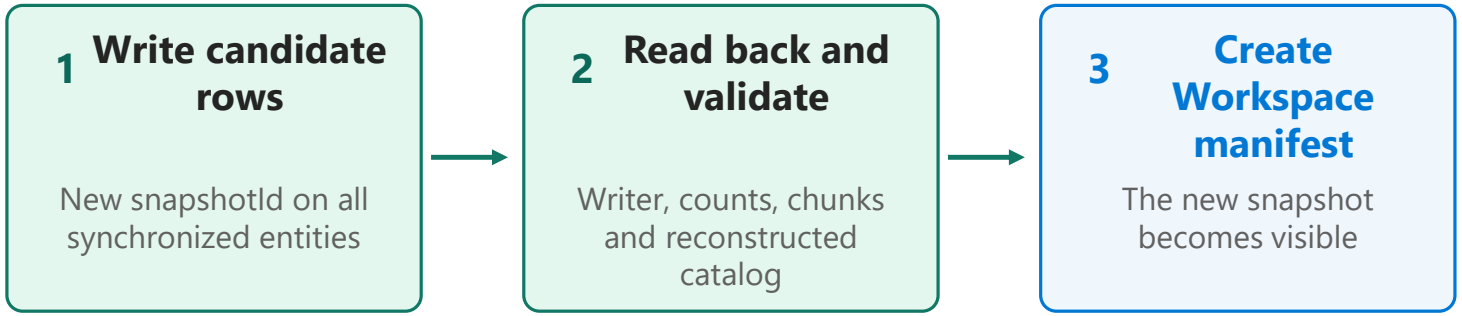



**Every invocation has bounded time, attempts, pages and response size.**
**Completed evidence is retained while remaining work continues.**

- Core creates the workspace and item plan.
- Rayfin collectors handle Ontology 2.0, Item Relations, KQL, SQL and Power BI.
- Python runs only the requested scanner, PBIR-Legacy and Kusto gaps.

Source: src/atlas/browser-collector-sync.ts · rayfin/functions/

# The Workspace manifest publishes only a snapshot Atlas can read back.




**Failure before the manifest → candidate rows stay invisible and readers keep the previous snapshot.**

- Child rows use one new snapshotId.
- Read-back uses the same path as application startup.
- Workspace is written last and acts as the visibility switch.
- Workspace Hub combines the manifest with persisted SyncRun status.

Source: docs/data-model.md · src/atlas/backend.ts

# Rayfin receives twenty-six entity classes across four lifetimes.

## Snapshot catalog

Workspace, FabricItem, LineageEdge, Principal, AccessGrant, JobRun and ConfigEntry

## Shared control state

Comment, policies, exceptions, WorkspaceScope and synchronizer authority

## Sync and evidence state

Sync runs, tasks, commands, payload records and evidence snapshots

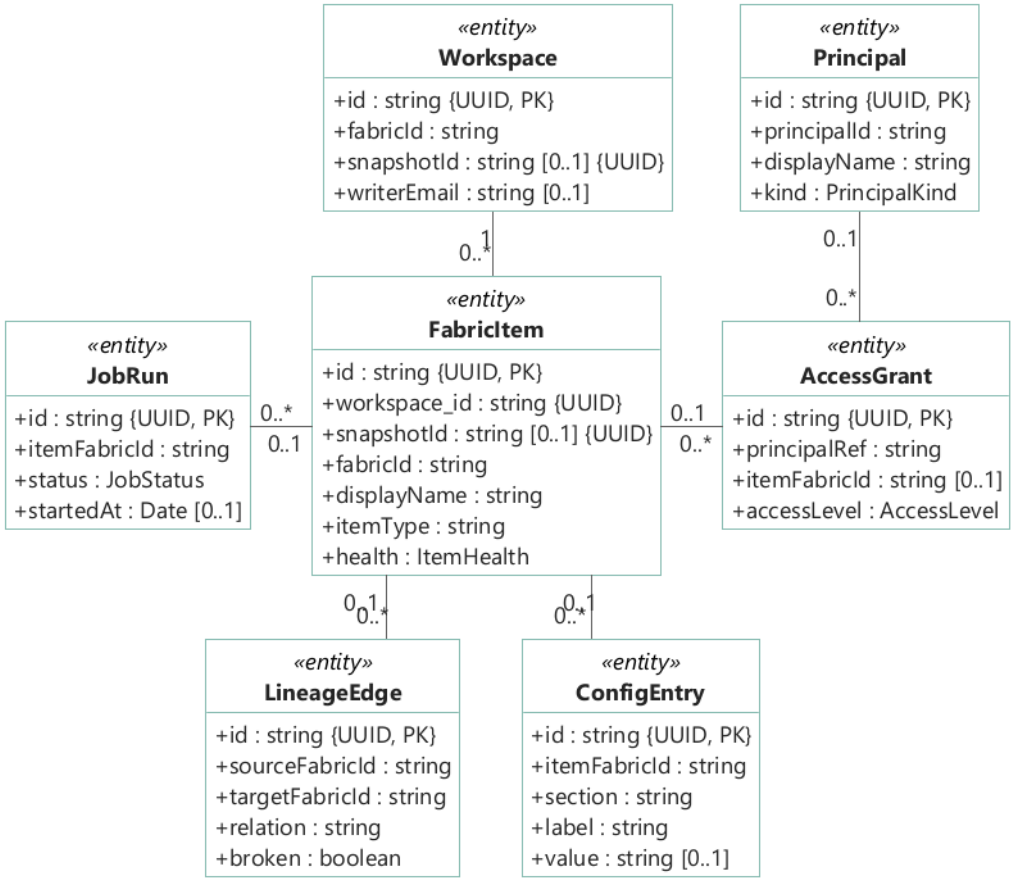
## Personal state

SavedView, AccessReview, AccessReviewEvent and FindingAck
are filtered by user_id.

**The entity classes define the database schema and the authorization boundary.**

Source: rayfin/data/schema.ts

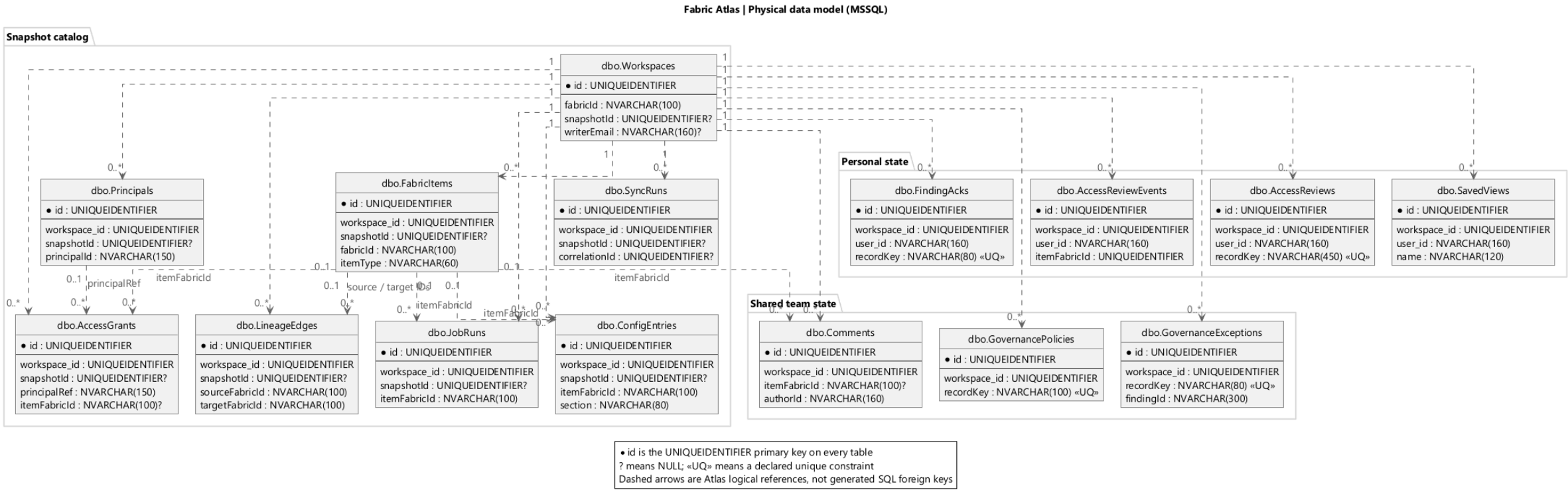
# The core UML shows snapshot entities and logical associations.




- Selected properties mirror the core TypeScript entities.
- Workspace publishes a manifest; FabricItem anchors item evidence.
- Additional v2 sync and evidence entities are grouped outside this core view.

Source: rayfin/data/*.ts (core entities)

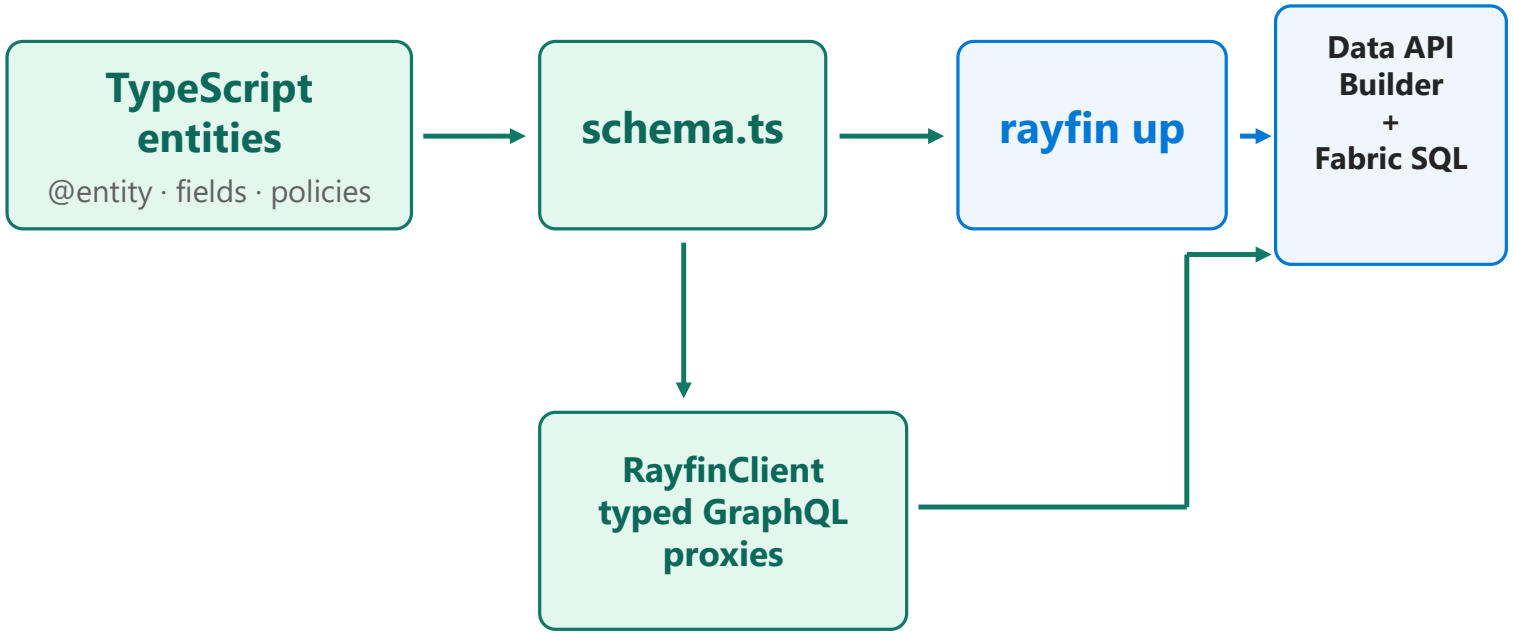
# The MPD shows the fifteen core MSSQL tables and logical links.




**Fabric Atlas 2.1 keeps eleven sync, scope and evidence tables outside this core view.**

Source: rayfin/data/schema.ts · @microsoft/rayfin-core 1.36.2

# Rayfin turns decorated TypeScript classes into the Fabric SQL data model.




- Field decorators define SQL types, limits and optional values.
- Authorization decorators generate read and write policies.
- schema.ts registers the 26 entities used by RayfinClient.

Source: rayfin/data/schema.ts · rayfin/rayfin.yml

# Three platform gaps keep the Python compatibility UDF.

1. Power BI application audience or first-party scanner connector.
2. Kusto application audience for live schema metadata.
3. PBIR-Legacy report-page coverage.
4. Once all three pass parity, delete sync_compatibility and the Python UDF.

```
Scheduling is separate: it needs an unattended trigger plus durable claim and publication fencing.
```

Source: docs/rayfin-platform-gaps.md

FABRIC ATLAS

# A trusted catalog stays available while the next snapshot is built.

Collect with Rayfin Functions. Keep exact compatibility gaps. Publish after read-back.

## Questions and feedback

github.com/fredgis/FabricAtlas

Rayfin Functions · Fabric SQL · Multi-workspace · Metadata only

Source: github.com/fredgis/FabricAtlas